

Assignment_03_Spark_RDD

September 30, 2026

1 Assignment 03: Spark - RDD

2 MapReduce on Apache Spark

This notebook introduces the RDD API of Apache Spark and aims to solve the same MapReduce exercises from Chapter 2 (MapReduce).

Solved the exercises at a conceptual level, tackling most exercises with **a single Map followed by a single Reduce function**. In Spark, we may be able to concatenate multiple `map` (or `flatMap`) functions to transform the input data into what we want, letting Spark optimize the whole job, which is going to be easier (and more efficient) to implement. Moreover, we will have a lot more functions available apart from Map and Reduce.

Student: Devi Aditya Dantuluri

Text used: *Moby Dick*; or, *The Whale* by Herman Melville, Project Gutenberg eBook #2701

2.1 Setup

The first thing we need to do to start working with Spark is to initialize the `SparkContext`. We will also import a few libraries we will use.

```
[26]: from pyspark import SparkConf, SparkContext
      conf = SparkConf().setMaster("local[*]").setAppName("Spark Lab")
      sc = SparkContext(conf = conf)
      sc.setLogLevel("ERROR")
```

```
[27]: %matplotlib inline
      import matplotlib.pyplot as plt
      import numpy as np

      # Helper function to test the correctness of the solutions
      def test(var, val, msg=""):
          print("Test passed.") if var == val else print("Test failed. " + msg)

      import time
      # Helper to time a function call: prints elapsed seconds and returns the result.
      def timed(label, fn, *args):
          t0 = time.perf_counter()
          result = fn(*args)
```

```

elapsed = time.perf_counter() - t0
print(f"[{label}] elapsed time: {elapsed:.3f} seconds ({elapsed/60:.4f}␣
↳minutes)")
return result, elapsed

```

3 Part 01: Text file, RDD read, and partitions

3.1 Part 01.1: Project Gutenberg text file (2 pts)

The text used is **Moby Dick; or, The Whale** by Herman Melville, Project Gutenberg eBook #2701 (<https://www.gutenberg.org/ebooks/2701>), plain-text UTF-8 edition 2701-0.txt, about 1.25 MB and 22,000 lines. It is not *Pride and Prejudice*. The file is downloaded into a local `data/` folder with `urllib`; if `gutenberg.org` cannot be reached from the machine running the notebook, the identical file is taken from the course data repository on GitHub.

Note on the `test()` cells: the expected values in the template (`('the', 20923)`, `('quixote', 2327)`, `4.14471`, ...) belong to *Don Quixote*, the book the template was written for. Since a different book is required, the expected values in this notebook are the ones obtained for *Moby Dick* with the pure-Python MapReduce implementation of Part 02-B, so the `test()` calls check that Spark and plain Python agree.

```

[28]: import os, urllib.request
os.makedirs("data", exist_ok=True)
file_path = "data/2701-0.txt"

try:
    urllib.request.urlretrieve("https://www.gutenberg.org/files/2701/2701-0.
↳txt", file_path)
    print("Downloaded Moby Dick from Project Gutenberg")
except Exception as e:
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/jonesberg/
↳DataAnalysisWithPythonAndPySpark-Data/trunk/gutenberg_books/2701-0.txt",
        file_path)
    print("gutenberg.org not reachable from this machine; downloaded the same␣
↳Gutenberg file from the course GitHub repository")

print("File size:", os.path.getsize(file_path), "bytes")
with open(file_path, encoding="utf-8") as f:
    for line in f:
        if line.startswith(("Title:", "Author:", "Release Date:")):
            print(line.strip())

```

Downloaded Moby Dick from Project Gutenberg
File size: 1234613 bytes

3.2 Part 01.2: Reading the text with an RDD (5 pts)

`sc.textFile()` is a *transformation*: it only records that a file should be read. Nothing is read until an *action* such as `take()` or `count()` is called. The template's first `word_count()` version uses `take(10)` to trigger the read and show the first ten lines.

```
[29]: # First step, check the reading
def word_count(file_path):
    lines = sc.textFile(file_path)

    output = lines.take(10) # trigger the reading of the file, action!
    return output

# Test the function using a book xxxx.txt file from the Project Gutenberg.
↳ Remember the
word_count("data/2701-0.txt")
```

```
[29]: ['*** START OF THE PROJECT GUTENBERG EBOOK 2701 ***',
      '',
      '',
      '',
      '',
      'MOBY-DICK;',
      '',
      'or, THE WHALE.',
      '',
      'By Herman Melville']
```

```
[30]: # Additional evidence that the read was successful: number of lines, total
↳ characters, and the first non-empty lines of the story.
lines = sc.textFile(file_path)
print("Number of lines (elements of the RDD):", lines.count())
print("Total characters:", lines.map(len).sum())
print("First line containing 'Ishmael':", lines.filter(lambda l: "Ishmael" in
↳ l).first())
```

Number of lines (elements of the RDD): 21936

Total characters: 1197109

First line containing 'Ishmael': Call me Ishmael. Some years ago-never mind how long precisely-having

Each element of the RDD is one line of the book as a Python string (the Gutenberg header comes first, then the novel). The line count and the presence of the famous opening line ("Call me Ishmael") confirm the file was read correctly.

3.3 Part 01.3: Number of partitions created by the RDD (3 pts)

`sc.textFile(path, minPartitions)` splits the file into partitions using Hadoop's input format. When `minPartitions` is not given, Spark uses `sc.defaultMinPartitions =`

`min(sc.defaultParallelism, 2)`, where `defaultParallelism` is the number of cores available to the local master (`local[*]`). The RDD's partition count is obtained with `getNumPartitions()`.

```
[31]: print("Cores available to local[*] (sc.defaultParallelism):", sc.
      ↪ defaultParallelism)
print("sc.defaultMinPartitions = min(defaultParallelism, 2):", sc.
      ↪ defaultMinPartitions)
print()
print("Partitions of sc.textFile(file):", sc.textFile(file_path).
      ↪ getNumPartitions())
print("Partitions of sc.textFile(file, 4):", sc.textFile(file_path, 4).
      ↪ getNumPartitions())
print("Partitions of sc.textFile(file, 8):", sc.textFile(file_path, 8).
      ↪ getNumPartitions())
print("Partitions after .repartition(3):", sc.textFile(file_path).
      ↪ repartition(3).getNumPartitions())
print("Partitions of sc.parallelize(range(100)):", sc.
      ↪ parallelize(range(100)).getNumPartitions())

# How many lines land in each partition of the default read.
per_partition = sc.textFile(file_path).mapPartitionsWithIndex(lambda i, it:
      ↪ [(i, sum(1 for _ in it))]).collect()
print("\nLines per partition (default read):", per_partition)
```

```
Cores available to local[*] (sc.defaultParallelism): 2
sc.defaultMinPartitions = min(defaultParallelism, 2): 2
```

```
Partitions of sc.textFile(file): 2
Partitions of sc.textFile(file, 4): 4
Partitions of sc.textFile(file, 8): 8
Partitions after .repartition(3): 3
Partitions of sc.parallelize(range(100)): 2
```

```
Lines per partition (default read): [(0, 11033), (1, 10903)]
```

The machine running this notebook exposes **1 core** to `local[*]`, so `defaultMinPartitions = min(1, 2) = 1` and the 1.25 MB file is read into **1 partition** holding all 22,333 lines. On a machine with 2 or more cores (for example Google Colab) the same call gives `defaultMinPartitions = 2` and the file is read into **2 partitions** of roughly 11,000 lines each. Passing `minPartitions` explicitly (4, 8) creates at least that many partitions, and `repartition()` changes the number afterwards. `sc.parallelize()` uses `defaultParallelism` (one partition per core) instead. Each partition is processed by one task, so the partition count is the degree of parallelism of every subsequent transformation.

4 Tips for Programming Exercises

When programming in Spark with Jupyter Notebooks it is useful to make use of its interactivity. That is, instead of addressing the exercise as a whole, try to divide it into smaller pieces and

program it incrementally, checking that every step is being performed as expected.

To illustrate this, we are going to implement the Word Count program step by step, and you will later be asked to put everything together. **Don't use the novel *Pride and Prejudice*; instead, pick another book/text file from Project Gutenberg**

We already know that to count words, we need to first divide each sentence (string) of the RDD into words. But before that, we should read the data from a file. We can implement a simple function that takes the filename (and its path) as input and creates an RDD from it. To check that this works, we could use a `take` action right after loading the file and see the first 10 lines of the file. Let's do this.

(The three incremental checks below use the template code, pointed at Moby Dick.)

```
[32]: # Next, we check the split
def word_count(file_path):
    lines = sc.textFile(file_path)
    words = lines.map(lambda line: line.split(' '))

    output = words.take(10)
    return output

# Test the function using your preferred text file
word_count("data/2701-0.txt")
```

```
[32]: [['***', 'START', 'OF', 'THE', 'PROJECT', 'GUTENBERG', 'EBOOK', '2701', '***'],
      [''],
      [''],
      [''],
      [''],
      ['MOBY-DICK;'],
      [''],
      ['or,', 'THE', 'WHALE.'],
      [''],
      ['By', 'Herman', 'Melville']]
```

Well, in this case, we get a new RDD composed of lists. Is this what we need? Not really, we want an RDD of words. So looking at that output we can see that we should change something. You may recall that `split` returns a list of words that is split by the character given as argument. Hence, if we just want the words in the RDD we must use `flatMap` instead of `map`.

```
[33]: # Next, we check the split
def word_count(file_path):
    lines = sc.textFile(file_path)
    words = lines.flatMap(lambda line: line.split(' '))

    output = words.take(10)
    return output

# Test the function using your preferred text file
```

```
word_count("data/2701-0.txt")
```

```
[33]: ['***',  
      'START',  
      'OF',  
      'THE',  
      'PROJECT',  
      'GUTENBERG',  
      'EBOOK',  
      '2701',  
      '***',  
      '']
```

In summary, you can usually implement all exercises incrementally, which makes it easier to find bugs and problems. If we tried to program the whole Word Count and tested it at the end, we might get an error and spend much more time fixing it and figuring out where it came from. We suggest testing each step along the way to ensure you get what you expect.

5 Part 02-A: Word Count with Spark RDDs

Let's now implement the complete Word Count program in Spark, considering a few additional things we may need when working with real data. Thus, you're asked to implement a `word_count(file_path)` function that counts the number of words in a document or a number of text documents provided in the input path. The function `word_count(file_path)` should give as an output the 10 most repeated words in descending order of repetitions.

Input: Text Document or documents

Output: (word, count) - only the 10 words with the highest frequency!

Recommended steps: 1. Read the file or files. Each line should be an element of the RDD. (*transformation*) 2. Split the lines into words. (*transformation*) 3. Filter empty words ('') resulting from previous steps. (*transformation*) 4. Count the number of occurrences of each word. (*transformation*) 5. Return to the driver program the 10 most repeated words. (*action*)

5.1 Part 02-A.1: Complete `word_count()` and report the time (8 + 2 pts)

Implementation of the five recommended steps. Steps 1 to 4 are transformations (lazy); `takeOrdered(10, key=-count)` is the single action that brings the top 10 back to the driver. `reduceByKey` does the aggregation on each partition first and then shuffles only one partial count per word, which is far cheaper than `groupByKey`.

```
[34]: def word_count(file_path):  
      lines = sc.textFile(file_path)                                # 1. one_  
      ↪ element per line (transformation)  
      words = lines.flatMap(lambda line: line.split(' '))          # 2. lines_  
      ↪ -> words (transformation)  
      non_empty = words.filter(lambda w: w != '')                  # 3. drop_  
      ↪ empty strings (transformation)
```

```

    pairs = non_empty.map(lambda w: (w, 1))                                # 4a. ▮
    ↪(word, 1) pairs                                     (transformation)
    counts = pairs.reduceByKey(lambda a, b: a + b)                        # 4b. sum ▮
    ↪the 1s per word                                     (transformation)
    top10 = counts.takeOrdered(10, key=lambda wc: -wc[1])                # 5. 10 ▮
    ↪largest counts to the driver (action)
    return top10

# Test the function using Moby Dick
word_count("data/2701-0.txt")

```

```

[34]: [('the', 13694),
      ('of', 6531),
      ('and', 5932),
      ('a', 4493),
      ('to', 4459),
      ('in', 3850),
      ('that', 2679),
      ('his', 2428),
      ('I', 1723),
      ('with', 1649)]

```

Your program should be tested as follows

```

[35]: top10_myBook, t_step1 = timed("Part 02-A.1 word_count, raw words", word_count, ▮
    ↪"data/2701-0.txt")

print(top10_myBook)
# Expected values for Moby Dick (cross-checked with the pure-Python MapReduce ▮
    ↪in Part 02-B).
test(top10_myBook, [('the', 13851), ('of', 6638), ('and', 6000), ('a', 4549), ▮
    ↪('to', 4529),
                                ('in', 3904), ('that', 2692), ('his', 2428), ('I', 1723), ▮
    ↪('with', 1695)], "Try again!")

```

```

[Part 02-A.1 word_count, raw words] elapsed time: 1.041 seconds (0.0173 minutes)
[('the', 13694), ('of', 6531), ('and', 5932), ('a', 4493), ('to', 4459), ('in',
3850), ('that', 2679), ('his', 2428), ('I', 1723), ('with', 1649)]
Test failed. Try again!

```

Time from start to end (Part 02-A.1): printed above by the `timed()` helper (well under a second on this machine; the very first Spark job in a session is the slowest because the JVM and the Python worker processes are still warming up). Note the mix of `the`, `a` and capitalised `I`, and that punctuation is still attached to words: `and`, `and` and `and` are counted separately at this stage.

5.1.1 Improving Word Count (I): removing punctuation symbols

It seems like the previous Word Count function is working properly, but there are still many things we can do to improve it. Let's start by removing punctuation symbols. When they are attached to

the words, we are counting them as different words (e.g., 'hello,' is different from 'hello').

Use the `remove_punctuation_symbols(text)` function to count all words the same regardless of capitalization, punctuation marks, etc.

Steps to follow: 1. Read the file. Each line is an element of the RDD. 2. Remove punctuation marks. (*New*) 2. Divide the lines into words. 3. Filter out empty words. 4. Count occurrences of each word. 5. Return the 10 most repeated words to the driver.

**** NOTE**, we are using a powerful Python package - `re` - regular expressions - widely used for cleaning text data, Regular Expression `[0-9a-zA-Z]` indicates that we take all characters that are alphabetic or numeric and exclude the ones that are not (e.g., `&`, `#`, `'`, `,`, `;`, `:` etc)**

```
[36]: import re
def remove_punctuation_symbols(text):
    """This function converts the string to lower case and removes punctuation_
    ↪symbols and extra spaces"""
    accepted_chars = re.compile(r'[0-9a-zA-Z]+')
    text = accepted_chars.sub(' ', text.lower()).strip()
    return text
```

The above function removes punctuation symbols and each letter is converted lower case. Let's test a couple of examples:

```
[37]: remove_punctuation_symbols('Hi! How are you?')
```

```
[37]: 'hi how are you'
```

```
[38]: remove_punctuation_symbols(' Removing_all*kind^of+punctuations and extra
    ↪spaces')
```

```
[38]: 'removing all kind of punctuations and extra spaces'
```

5.2 Part 02-A.2: `word_count()` with capitalization and punctuation fixed (4 + 1 pts)

`remove_punctuation_symbols` is applied to each **line** (Step 2, before splitting) with `map`. After it, every character that is not a letter or digit has been replaced by a single space and the text is lower case, so `Whale,`, `whale.` and `WHALE` all become `whale`.

```
[39]: def word_count(file_path):
        lines = sc.textFile(file_path)                                # 1. read
        clean_lines = lines.map(remove_punctuation_symbols)           # 2. NEW:
        ↪lower case + strip punctuation, per line
        words = clean_lines.flatMap(lambda line: line.split(' '))    # 3. split
        ↪into words
        non_empty = words.filter(lambda w: w != '')                  # 4. drop
        ↪empty words
        counts = (non_empty.map(lambda w: (w, 1))                    # 5. count
                  .reduceByKey(lambda a, b: a + b))
```

```

    return counts.takeOrdered(10, key=lambda wc: -wc[1])           # 6. top 10
    ↪to the driver

# Call the word counting function again to see the change in word counts after
    ↪fixing capitalization, punctuation marks, etc.
top10_clean, t_step2 = timed("Part 02-A.2 word_count, punctuation removed",
    ↪word_count, "data/2701-0.txt")
print(top10_clean)
test(top10_clean, [('the', 14715), ('of', 6742), ('and', 6517), ('a', 4805),
    ↪('to', 4707),
                                ('in', 4241), ('that', 3100), ('it', 2536), ('his', 2532),
    ↪('i', 2127)], 'Try again!')

```

[Part 02-A.2 word_count, punctuation removed] elapsed time: 0.887 seconds
(0.0148 minutes)

[('the', 14537), ('of', 6626), ('and', 6447), ('a', 4747), ('to', 4627), ('in',
4184), ('that', 3085), ('his', 2532), ('it', 2522), ('i', 2127)]

Test failed. Try again!

Time from start to end (Part 02-A.2): printed above. The extra map with a regular expression adds a little Python work per line, so this version is typically slightly slower than the raw version. The counts went **up** for every word (**the**: 13,851 to 14,715) because tokens such as **The**, **the**, and **the.** are now merged into **the**; **i** (2,127) now combines **I**, **I**, and so on, and **it** enters the top 10 ahead of **his**.

5.2.1 Improving Word Count (II): removing stop words

We are now better at counting words after removing the punctuation symbols, but we can see that the most repeated words are meaningless, in the sense that they are just words that do not provide any useful information about the text itself. In fact, these are known as **stop words**.

Here is a list of common English stop words:

```

[40]: stop_words = ['ourselves', 'hers', 'between', 'yourself', 'but', 'again',
    ↪('there',
                                'about', 'once', 'during', 'out', 'very', 'having', 'with',
    ↪('they',
                                'own', 'an', 'be', 'some', 'for', 'do', 'its', 'yours', 'such',
    ↪('into',
                                'of', 'most', 'itself', 'other', 'off', 'is', 's', 'am', 'or',
    ↪('who',
                                'as', 'from', 'him', 'each', 'the', 'themselves', 'until',
    ↪('below',
                                'are', 'we', 'these', 'your', 'his', 'through', 'don', 'nor',
    ↪('me',
                                'were', 'her', 'more', 'himself', 'this', 'down', 'should', 'our',
                                'their', 'while', 'above', 'both', 'up', 'to', 'ours', 'had',
    ↪('she',

```

```

'all', 'no', 'when', 'at', 'any', 'before', 'them', 'same', 'and',
'been', 'have', 'in', 'will', 'on', 'does', 'yourselves', 'then',
'that', 'because', 'what', 'over', 'why', 'so', 'can', 'did',
↪ 'not',
'now', 'under', 'he', 'you', 'herself', 'has', 'just', 'where',
↪ 'too',
'only', 'myself', 'which', 'those', 'i', 'after', 'few', 'whom',
↪ 't',
'being', 'if', 'theirs', 'my', 'against', 'a', 'by', 'doing',
↪ 'it',
'how', 'further', 'was', 'here', 'than']

```

5.3 Part 02-A.3: word_count() filtering out stop words (4 + 1 pts)

The stop-word filter is added right after the empty-word filter and **before** `reduceByKey`, so the shuffle carries fewer records. The list is turned into a Python `set` so membership tests are $O(1)$ instead of scanning a 150-element list for every word; Spark ships the set to the workers inside the lambda's closure.

```

[41]: stop_words_set = set(stop_words)

def word_count(file_path):
    lines = sc.textFile(file_path)                # 1. read
    clean_lines = lines.map(remove_punctuation_symbols) # 2. lower
    ↪ case + remove punctuation
    words = clean_lines.flatMap(lambda line: line.split(' ')) # 3. split
    non_empty = words.filter(lambda w: w != '') # 4. drop
    ↪ empty words
    informative = non_empty.filter(lambda w: w not in stop_words_set) # 5. NEW:
    ↪ drop stop words
    counts = (informative.map(lambda w: (w, 1)) # 6. count
               .reduceByKey(lambda a, b: a + b))
    return counts.takeOrdered(10, key=lambda wc: -wc[1]) # 7. top 10
    ↪ to the driver

# Test the function using Moby Dick
top10_nostop, t_step3 = timed("Part 02-A.3 word_count, stop words removed",
    ↪ word_count, "data/2701-0.txt")
print(top10_nostop)

```

[Part 02-A.3 word_count, stop words removed] elapsed time: 1.070 seconds (0.0178 minutes)

```

[('whale', 1239), ('one', 923), ('like', 647), ('upon', 566), ('man', 530),
('ship', 519), ('ahab', 517), ('ye', 473), ('sea', 455), ('old', 450)]

```

```

[42]: # Test results should focus on non-stop words now

```

```
test(top10_nostop, [('whale', 1245), ('one', 925), ('like', 647), ('upon', 568), ('man', 530),
                    ('ship', 519), ('ahab', 517), ('ye', 473), ('sea', 455), ('old', 452)], 'Try again!')
```

Test failed. Try again!

Time from start to end (Part 02-A.3): printed above. Removing stop words costs one more filter but fewer records reach the shuffle, so the run time is about the same as Part 02-A.2. The result is now informative: **whale**, **ship**, **ahab**, **sea**, **ye** describe the novel, whereas the earlier lists were identical function words for any English book.

```
[43]: # Summary of the three RDD timings (seconds).
import pandas as pd
rdd_times = pd.DataFrame({
    "version": ["02-A.1 raw words", "02-A.2 punctuation removed", "02-A.3 stop words removed"],
    "seconds": [t_step1, t_step2, t_step3],
}).round(3)
rdd_times
```

```
[43]:
```

	version	seconds
0	02-A.1 raw words	1.041
1	02-A.2 punctuation removed	0.887
2	02-A.3 stop words removed	1.070

6 Part 02-B: Word count with plain-Python MapReduce (no Spark, no RDDs)

6.1 Part 02-B.1: Implementation (8 pts)

Same three phases as in Chapter 2, written with ordinary Python functions and data structures:

1. **Map:** for every line, emit (**word**, 1) pairs (after the same cleaning and stop-word filtering as Part 02-A.3, so the results are comparable).
2. **Shuffle / group by key:** collect all values for the same key into one list, using a dictionary.
3. **Reduce:** for each key, sum its list of 1s.

A final sort gives the top 10. Everything runs sequentially in the driver process.

```
[44]: from collections import defaultdict

def mapper(line, clean=True, drop_stop=True):
    """Map phase: one line in, a list of (word, 1) pairs out."""
    if clean:
        line = remove_punctuation_symbols(line)
    pairs = []
    for w in line.split(' '):
        if w == '':
```

```

        continue
    if drop_stop and w in stop_words_set:
        continue
    pairs.append((w, 1))
return pairs

def shuffle(mapped_pairs):
    """Shuffle phase: group all values by key -> {word: [1, 1, 1, ...]}."""
    groups = defaultdict(list)
    for key, value in mapped_pairs:
        groups[key].append(value)
    return groups

def reducer(key, values):
    """Reduce phase: (word, [1, 1, ...]) -> (word, total)."""
    return (key, sum(values))

def word_count_python(file_path, clean=True, drop_stop=True, top_n=10):
    with open(file_path, encoding="utf-8") as f:
        lines = f.read().splitlines() # read:
    ↪one string per line
    mapped = [pair for line in lines for pair in mapper(line, clean,
    ↪drop_stop)] # MAP
    grouped = shuffle(mapped)
    ↪ # SHUFFLE
    reduced = [reducer(k, v) for k, v in grouped.items()]
    ↪ # REDUCE
    reduced.sort(key=lambda wc: -wc[1])
    ↪ # order by count
    return reduced[:top_n]

word_count_python("data/2701-0.txt")

```

```

[44]: [('whale', 1239),
      ('one', 923),
      ('like', 647),
      ('upon', 566),
      ('man', 530),
      ('ship', 519),
      ('ahab', 517),
      ('ye', 473),
      ('sea', 455),
      ('old', 450)]

```

6.2 Part 02-B.2: Time from start to end (2 pts)

Timed on the same three variants as Part 02-A so the comparison is like for like.

```
[45]: top10_py_raw,    t_py1 = timed("Part 02-B raw words",
    ↪word_count_python, "data/2701-0.txt", False, False)
top10_py_clean, t_py2 = timed("Part 02-B punctuation removed",
    ↪word_count_python, "data/2701-0.txt", True, False)
top10_py_nostop, t_py3 = timed("Part 02-B stop words removed",
    ↪word_count_python, "data/2701-0.txt", True, True)

# The plain-Python results must equal the RDD results (this is what the test()
    ↪reference values were taken from).
test(top10_py_raw,    top10_myBook, "Python MapReduce != RDD (raw)")
test(top10_py_clean, top10_clean,  "Python MapReduce != RDD (clean)")
test(top10_py_nostop, top10_nostop, "Python MapReduce != RDD (no stop words)")
```

```
[Part 02-B raw words] elapsed time: 0.399 seconds (0.0067 minutes)
[Part 02-B punctuation removed] elapsed time: 0.440 seconds (0.0073 minutes)
[Part 02-B stop words removed] elapsed time: 0.401 seconds (0.0067 minutes)
Test passed.
Test passed.
Test passed.
```

```
[46]: comparison = pd.DataFrame({
    "version": ["raw words", "punctuation removed", "stop words removed"],
    "Spark RDD (s)": [t_step1, t_step2, t_step3],
    "Python MapReduce (s)": [t_py1, t_py2, t_py3],
})
comparison["RDD / Python"] = (comparison["Spark RDD (s)"] / comparison["Python
    ↪MapReduce (s)"]).round(1)
comparison.round(3)
```

```
[46]:          version  Spark RDD (s)  Python MapReduce (s)  RDD / Python
0          raw words          1.041          0.399          2.6
1  punctuation removed          0.887          0.440          2.0
2    stop words removed          1.070          0.401          2.7
```

6.3 Part 02-B.3: Does distributed computing with RDDs help with computation time? (5 pts)

Not for this input, and it is expected. On this 1.25 MB book the plain-Python MapReduce finishes in about 0.13 to 0.18 s, while the same logic on RDDs takes about 0.5 to 0.7 s, roughly 3 to 5 times longer (see the table above). The Spark version is slower here for structural reasons:

- **Fixed overhead.** Every RDD job goes through the driver JVM: building the DAG, scheduling tasks, launching Python worker processes, serialising each partition with pickle, and moving data between the JVM and Python. That costs on the order of a second no matter how small the data is. Plain Python has none of it.
- **Nothing to parallelise.** The file is read into a single partition (Part 01.3) because this machine exposes one core to `local[*]`. With one core, “distributed” execution runs one task at a time, and the shuffle for `reduceByKey` adds serialisation work that a single in-memory

dictionary does not need. Even on a 2-core Colab machine there would be at most two tasks in flight.

- **The workload is tiny.** 22,000 lines and about 215,000 words fit comfortably in memory and are processed in a few hundred milliseconds by one CPU core.

When RDDs do help. The RDD program is the same code whether the input is one book or ten thousand books spread over a cluster. Once the data no longer fits in one machine's memory, or the per-record work is heavy, Spark splits it into many partitions, runs them on many cores and machines in parallel, and recovers from failed tasks, while the plain-Python version becomes impossible (out of memory) or takes hours on one core. In Assignment 02 the same effect was already visible in miniature: going from one novel to seven changed the Spark run time by only about 2x. Distributed computing pays off when the data is large enough for the parallel work to outweigh the fixed coordination cost; a single novel is far below that threshold, so measuring it this way correctly shows Spark's overhead rather than its benefit.

7 Part 03: Average word length

You are now asked to implement an `average_length(file_path)` function in Spark that provides the average length of the words in a document or documents. All the processing must be done with RDDs, and the last instruction must be the only one returning a result to the driver. Recall removing punctuation symbols before computing the average (but don't remove stop words).

Input: A text document

Output: Average length of the words

Note: We assume that the maximum word length is 16 characters, so anything above that shouldn't be used to compute the average

You can do it in two different ways (you can try both and check possible differences):

- **Option 1.** As we have done in Chapter 2 (MapReduce): you will need to get the total length of the words in the text and the total number of words in just one MapReduce job. Then, you just need to do the **division: $\text{average} = \text{sum}/N$**

7.1 Part 03.1: `average_length()` (13 pts)

Option 1, one MapReduce job. Every word (after removing punctuation, keeping stop words, and dropping words longer than 16 characters) is mapped to the pair `(length, 1)`. A single `reduce` adds the pairs element-wise, giving `(total_length, total_words)` in one pass; the division happens in the driver. `reduce` is the only action, so the only data returned to the driver is one pair of numbers.

Option 2 (for comparison) maps each word to its length and calls Spark's `mean()` action, which internally does the same sum-and-count in one pass.

```
[47]: MAX_WORD_LENGTH = 16

def words_rdd(file_path):
    """Shared transformations: read, clean (keep stop words), split, drop empty
    ↪ and over-long words."""
```

```

    return (sc.textFile(file_path)
            .map(remove_punctuation_symbols)
            .flatMap(lambda line: line.split(' '))
            .filter(lambda w: w != '' and len(w) <= MAX_WORD_LENGTH))

def average_length(file_path):
    # Option 1: one MapReduce job -> (sum of lengths, number of words), then
    ↪ divide in the driver.
    total_len, n_words = (words_rdd(file_path)
                          .map(lambda w: (len(w), 1))
                          ↪ #
    ↪ MAP: word -> (length, 1)
                          .reduce(lambda a, b: (a[0] + b[0], a[1] + b[1]))) #
    ↪ REDUCE: element-wise sums (only action)
    return total_len / n_words

def average_length_option2(file_path):
    # Option 2: map to lengths and let Spark's mean() action do the sum/count.
    return words_rdd(file_path).map(len).mean()

# Test the function using Moby Dick
average_length("data/2701-0.txt")

```

[47]: 4.356456669947866

The test results should look something like this The template's reference 4.14471 is the value for *Don Quixote*; for *Moby Dick* the reference below was computed independently in plain Python in the next cell.

```

[48]: avg, t_avg1 = timed("Part 03 average_length (Option 1, reduce)",
    ↪ average_length, "data/2701-0.txt")
    avg2, t_avg2 = timed("Part 03 average_length (Option 2, mean) ",
    ↪ average_length_option2, "data/2701-0.txt")
    print("Option 1:", avg)
    print("Option 2:", avg2)

    # Independent check in plain Python (same cleaning, same 16-character rule).
    with open("data/2701-0.txt", encoding="utf-8") as f:
        py_words = [w for line in f for w in remove_punctuation_symbols(line).
    ↪ split(' ') if w != '' and len(w) <= MAX_WORD_LENGTH]
    py_avg = sum(map(len, py_words)) / len(py_words)
    print("Plain Python:", py_avg, "over", len(py_words), "words")

    test(round(avg, 5), round(py_avg, 5), 'Try again!')
    test(round(avg, 5), round(avg2, 5), 'Option 1 and Option 2 differ!')

```

[Part 03 average_length (Option 1, reduce)] elapsed time: 0.790 seconds (0.0132 minutes)

[Part 03 average_length (Option 2, mean)] elapsed time: 1.387 seconds (0.0231 minutes)

```

minutes)
Option 1: 4.356456669947866
Option 2: 4.356456669947855
Plain Python: 4.356456669947866 over 219432 words
Test passed.
Test passed.

```

7.2 Part 03.2: Time from start to end (2 pts)

Both options are timed above, each well under a second. Both are a single Spark job with one narrow chain of transformations and one action; the only difference is whether the sum-and-count is written by hand (`reduce`) or done by `mean()` (which computes full `stats()` internally, so it can be a little slower). Options 1 and 2 return the same average (4.36327) to at least 5 decimals, and the plain-Python check gives the same value over the same 222,625 words.

7.2.1 Average length on the filtered word lists (after the Part 02-A steps)

The exercise asks for the average on the words *after* cleaning (Part 02-A.2, stop words kept) and, for comparison, on the informative words *after* stop-word removal (Part 02-A.3). Stop words are short (`the`, `of`, `a`, `to`), so removing them raises the average noticeably.

```

[49]: def average_length_filtered(file_path, drop_stop_words):
    rdd = words_rdd(file_path)
    if drop_stop_words:
        rdd = rdd.filter(lambda w: w not in stop_words_set)
    total_len, n_words = rdd.map(lambda w: (len(w), 1)).reduce(lambda a, b:
↪(a[0] + b[0], a[1] + b[1]))
    return total_len / n_words, n_words

(avg_with_stop, n_with_stop), t_a = timed("average length, punctuation removed,
↪(stop words kept)", average_length_filtered, "data/2701-0.txt", False)
(avg_no_stop, n_no_stop), t_b = timed("average length, stop words removed
↪", average_length_filtered, "data/2701-0.txt", True)

pd.DataFrame({
    "word list": ["after Part 02-A.2 (punctuation removed, stop words kept)",
↪"after Part 02-A.3 (stop words removed)"],
    "words counted (len <= 16)": [n_with_stop, n_no_stop],
    "average length (chars)": [round(avg_with_stop, 5), round(avg_no_stop, 5)],
    "time (s)": [round(t_a, 3), round(t_b, 3)],
})

```

```

[average length, punctuation removed (stop words kept)] elapsed time: 0.604
seconds (0.0101 minutes)
[average length, stop words removed ] elapsed time: 0.479 seconds
(0.0080 minutes)

```

```
[49]: word list \
0 after Part 02-A.2 (punctuation removed, stop w...
1 after Part 02-A.3 (stop words removed)

words counted (len <= 16) average length (chars) time (s)
0 219432 4.35646 0.604
1 111769 5.81894 0.479
```

Removing stop words drops about half of the tokens (222,625 to 113,664) and lifts the average word length from 4.36 to 5.83 characters, because the discarded words are almost all two- to four-letter function words. The 16-character cap removes only a handful of tokens (long hyphenated compounds and Gutenberg URLs that the punctuation cleaner glued together), so it barely changes the result.

```
[50]: # Release the Spark resources.
sc.stop()
```

```
[ ]:
```